



MÜHENDİSLİK FAKÜLTESİ

SİBER GÜVENLİK

TEZSİZ YÜKSEK LİSANS DÖNEM PROJESİ

FARKLI İSTEMCİ TÜRLERİ TARAFINDAN
KULLANILAN WEB API'LERDE KİMLİK
DOĞRULAMA VE YETKİLENDİRME MODELLERİNİN
GÜVENLİK AÇISINDAN KARŞILAŞTIRMALI
ANALİZİ

AHMET YESEVİ
ÜNİVERSİTESİ

HAZIRLAYAN
Furkan Cemal ÇALIŞKAN

DANIŞMAN ÖĞRETİM ÜYESİ
Doç. Dr. Muharrem Tuncay GENÇOĞLU

ETİK İLKELERE UYGUNLUK BEYANI

Dönem projesi yazma sürecinde bilimsel ve etik ilkelere uyduğumu, yararlandığım tüm kaynakları kaynak gösterme ilkelerine uygun olarak kaynakçada belirttiğimi ve bu bölümler dışındaki tüm ifadelerin şahsıma ait olduğunu beyan ederim.



Furkan Cemal
ÇALIŞKAN

FARKLI İSTEMCİ TÜRLERİ TARAFINDAN KULLANILAN WEB API'LERDE KİMLİK DOĞRULAMA VE YETKİLENDİRME MODELLERİNİN GÜVENLİK AÇISINDAN KARŞILAŞTIRMALI ANALİZİ

Furkan Cemal ÇALIŞKAN

AHMET YESEVİ ÜNİVERSİTESİ

SİBER GÜVENLİK

2026

ÖZET

Bu araştırmada, web, mobil, masaüstü ve komut satırı uygulamaları tarafından kullanılan web API'lerinde kullanılan yetkilendirme modelleri ve kimlik doğrulama yöntemlerinin güvenlik açısından karşılaştırmalı analizi sunulmuştur. Günümüzde bu tür çoklu istemci mimarilerinin sektörler arası yaygınlaşmasıyla birlikte, her istemcinin kendine özel dinamikleri ve saldırı yüzeyleri olduğu için, her istemci türüne yönelik özel güvenlik önlemlerinin alınması gerekliliği doğmuştur. Bu bağlamda farklı kimlik doğrulama yöntemlerinin (JWT, OAuth 2.0, API anahtarı, Basic Authentication) nasıl çalıştığı ve bu yöntemlerin farklı istemci türleri üzerindeki etkileri karşılaştırmalı olarak analiz edilmiştir. Araştırmada, kimlik doğrulama yöntemlerinin test edilmesi amacıyla API ve diğer istemciler için ayrı ayrı projeler geliştirilmiştir. Bu projeler üzerinde erişim token'ları ve yenileme token'ları yapıları detaylı olarak incelenmiştir. Ayrıca bu yöntemler, yaygın güvenlik açıkları (XSS, CSRF, token hırsızlığı, replay saldırıları) açısından birbirleriyle karşılaştırılmıştır. Elde edilen bulgulara göre, güvenlik açıklarının istemci türüne ve kimlik doğrulama yöntemlerine göre farklılık gösterdiği tespit edilmiştir. Ayrıca her yöntemin farklı riskler barındırdığına ulaşılmıştır. Bu sonuçlar doğrultusunda, farklı uygulamalar arasında kurulacak güvenlik mimarileri için uygun kimlik doğrulama yöntemi seçimi konusunda öneriler sunulmuştur.

Anahtar Kelimeler: Kimlik doğrulama, yetkilendirme, web API güvenliği, JWT, OAuth 2.0, çoklu istemci mimarisi, güvenlik zafiyetleri

Danışman: Doç. Dr. Muharrem Tuncay GENÇOĞLU

COMPARATIVE ANALYSIS OF AUTHENTICATION AND AUTHORIZATION MODELS IN TERMS OF SECURITY FOR WEB APIs USED BY DIFFERENT CLIENT TYPES

Furkan Cemal ÇALIŞKAN

AHMET YESEVI UNIVERSITY

CYBER SECURITY

2026

ABSTRACT

This study conducts a comparison of authorization models and authentication types used in web APIs that support web, mobile, desktop, and command-line applications, focusing on security aspects. The increasing use of these multi-client architectures across different industries, along with the distinct characteristics and potential security risks associated with each type of client, has led to the need for security measures tailored to each client. In this context, various authentication methods, including JWT, OAuth 2.0, API key, and Basic Authentication, have been examined in terms of how they function and their impact on different client types. To test these authentication methods, separate projects were developed for APIs and other clients, and the structure of access tokens and refresh tokens was analyzed within these setups. These methods were evaluated in relation to common security issues such as XSS, CSRF, token theft, and replay attacks. The findings indicate that the presence and nature of vulnerabilities vary depending on both the client type and the authentication method used. Additionally, each method is associated with different levels of risk. Based on these results, recommendations are provided for choosing the most suitable authentication method when designing a secure architecture for application interactions.

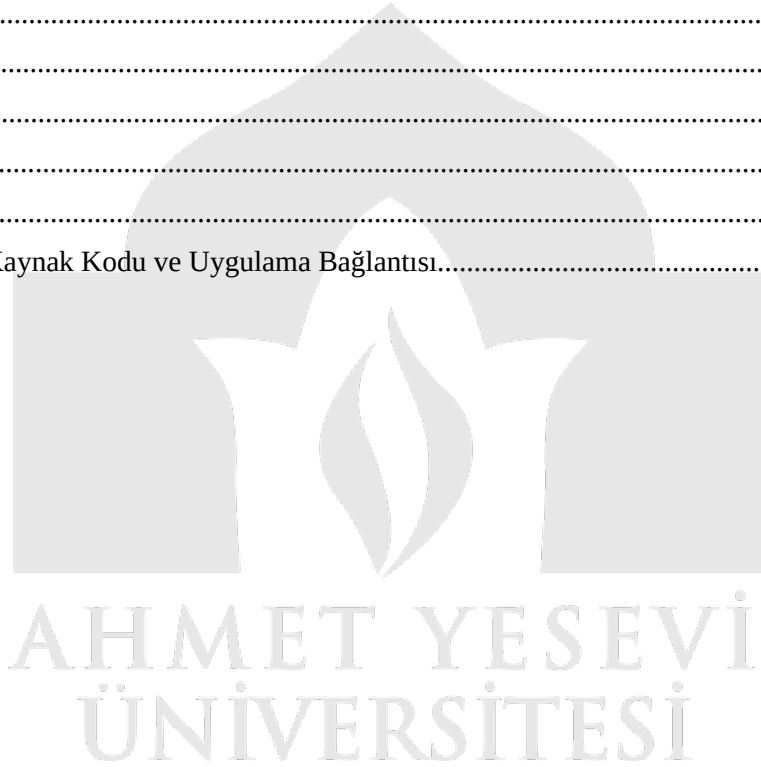
Keywords: Authentication, authorization, web API security, JWT, OAuth 2.0, multi-client architecture, security vulnerabilities

Advisor: Doç. Dr. Muharrem Tuncay GENÇOĞLU

İÇİNDEKİLER

ETİK İLKELERE UYGUNLUK BEYANI.....	ii
ÖZET.....	iii
ABSTRACT.....	iv
İÇİNDEKİLER.....	v
SİMGELER VE KISALTMALAR.....	vii
BÖLÜM I GİRİŞ.....	1
1.1. Problem.....	1
1.2. Araştırmanın Amacı.....	1
1.3. Araştırmanın Önemi.....	2
1.4. Sayıtlar.....	2
1.5. Sınırlılıklar.....	3
1.6. Tanımlar.....	3
BÖLÜM II KAVRAMSAL ÇERÇEVE.....	4
2.1. Kimlik Doğrulama ve Yetkilendirme Kavramları.....	4
2.2. Kimlik Doğrulama Yöntemleri.....	4
2.2.1. Oturum Tabanlı Kimlik Doğrulama (Session-Based Authentication).....	5
2.2.2. JSON Web Token (JWT).....	5
2.2.3. OAuth 2.0.....	5
2.2.4. API Anahtarı (API Key).....	6
2.2.5. Basic Authentication.....	6
2.3. Token Yapıları.....	6
2.3.1. Access Token.....	6
2.3.2. Refresh Token.....	7
2.4. İstemci Türleri.....	7
2.4.1. Web Uygulamaları.....	7
2.4.2. Mobil Uygulamalar.....	7
2.4.3. Masaüstü Uygulamaları.....	7
2.4.4. Komut Satırı Uygulamaları (CLI).....	8
2.5. Güvenlik Tehditleri.....	8
2.5.1. Cross-Site Scripting (XSS).....	8
2.5.2. Cross-Site Request Forgery (CSRF).....	8
2.5.3. Token Hırsızlığı (Token Theft).....	8
2.5.4. Replay Attack.....	8

BÖLÜM III YÖNTEM.....	9
3.1. Araştırmanın Modeli.....	9
3.2. Evren ve Örneklem.....	9
3.3. Veri Toplama Araçları.....	10
3.4. Verilerin Toplanması.....	10
3.5. Verilerin Analizi.....	11
BÖLÜM IV BULGULAR VE YORUM.....	12
4.1. Kimlik doğrulama yöntemlerine ilişkin bulgular.....	12
4.2. İstemci türlerine ilişkin bulgular.....	13
4.3. Güvenlik tehditlerine ilişkin bulgular.....	14
BÖLÜM V SONUÇ, TARTIŞMA VE ÖNERİLER.....	16
5.1. Sonuç.....	16
5.2. Tartışma.....	17
5.3. Öneriler.....	18
KAYNAKÇA.....	19
EKLER.....	20
EK A: Proje Kaynak Kodu ve Uygulama Bağlantısı.....	20



SİMGELER VE KISALTMALAR

API : Application Programming Interface (Uygulama Programlama Arayüzü)

JWT : JSON Web Token (JSON Web Belirteci)

OAuth : Open Authorization (Açık Yetkilendirme)

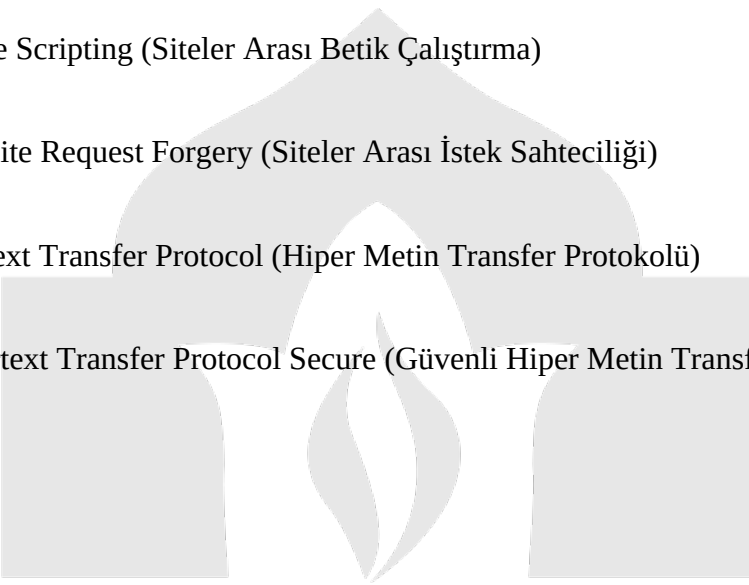
CLI : Command Line Interface (Komut Satırı Arayüzü)

XSS : Cross-Site Scripting (Siteler Arası Betik Çalıştırma)

CSRF : Cross-Site Request Forgery (Siteler Arası İstek Sahteciliği)

HTTP : Hypertext Transfer Protocol (Hiper Metin Transfer Protokolü)

HTTPS : Hypertext Transfer Protocol Secure (Güvenli Hiper Metin Transfer Protokolü)



AHMET YESEVİ
ÜNİVERSİTESİ

BÖLÜM I

GİRİŞ

1.1. Problem

Bir API tarafından birden fazla istemci türüne hizmet veren sistemlerde, kimlik doğrulama ve yetkilendirme süreçlerinin güvenli bir şekilde yönetilmesi oldukça önemlidir. Her istemci türü kendi özel veri işleme, saklama ve koruma yöntemlerine sahip olabilir. Bu nedenle sistemde güvenlik açısından farklılıklar oluşmaktadır. Bu farklılıklar, tek bir kimlik doğrulama veya yetkilendirme yöntemiyle güvenlik süreçlerinin yürütülmesini zorlaştırmaktadır. Bu yüzden istemci türüne göre uygun güvenlik tedbirlerinin seçilmesi, önemli bir konu olarak karşımıza çıkmaktadır. Yaygın yöntemler olarak kullanılan kimlik doğrulama teknikleri, örneğin JWT, OAuth 2.0 ve API anahtarları, yanlış kullanıldığında sistemin token hırsızlığı ve yetkisiz erişim gibi saldırılarla karşı karşıya kalmasına neden olmaktadır. Bu yüzden istemci türüne ve sistem durumuna uygun güvenlik önlemlerinin belirlenmesi oldukça önemli bir araştırma konusu olarak öne çıkmaktadır.

1.2. Araştırmanın Amacı

Bu çalışmanın temel hedefi, web API'lerinde farklı istemci türlerinin kullandığı kimlik doğrulama ve yetkilendirme modellerini güvenlik açısından karşılaştırmalı olarak incelemektir. Bu bağlamda, oturum tabanlı kimlik doğrulama, JWT, OAuth 2.0, API anahtarları ve temel kimlik doğrulama gibi yaygın yöntemlerin çalışma şekilleri analiz edilmekte ve bu yöntemlerin farklı istemci türlerine ne şekilde etki ettiği değerlendirilmektedir. Ayrıca, erişim belirteci ve yenileme belirteci yapıları incelenerek bu belirteçlerin güvenlik açısından hangi avantaj ve dezavantajları olduğu kısmen açıklanmaktadır. Aynı zamanda, her yöntemin XSS, CSRF, belirteç hırsızlığı ve tekrar oynatma saldırıları gibi yaygın güvenlik tehditlerine karşı nasıl dayanıklı olduğu da analiz edilmektedir. Çalışma, her istemci türü için en uygun kimlik doğrulama ve yetkilendirme yöntemlerine dair öneriler sunmayı hedeflemektedir.

1.3. Araştırmanın Önemi

Modern yazılım sistemlerinde çoklu istemci mimarilerinin yaygın kullanımı, güvenli kimlik doğrulama ve yetkilendirme mekanizmalarının önemi artırmıştır. Farklı platformlardan aynı API'ye erişim imkanı, güvenlik açıklarının daha geniş bir etki alanına yayılmasına neden olmaktadır. Bu sebeple, sistem geliştiricileri ve tasarımcılara uygun bir kimlik doğrulama yönteminin seçilmesi büyük önem taşımaktadır. Bu çalışma, istemci türüne göre farklı kimlik doğrulama yöntemlerini güvenlik açısından karşılaştırmalı olarak değerlendirmeyi amaçlamaktadır. Böylece literatürdeki bu konuda bir boşluk doldurulmaya çalışılmaktadır. Ayrıca, profesyonellere güvenli sistemlerin geliştirilmesinde faydalı pratik öneriler sunarak da önemli bir katkı sağlanmaktadır.

1.4. Sayıtlar

Bu çalışmada aşağıdaki varsayımlar yapılmıştır:

- Çalışmada incelenen kimlik doğrulama yöntemlerinin standartlara uygun şekilde uygulandığı kabul edilmiştir.
- Kullanılan istemci uygulamalarının temel düzeyde güvenlik önlemlerine sahip olduğu varsayılmıştır.
- Analiz edilen güvenlik tehditlerinin yaygın ve güncel saldırı türlerini temsil ettiği düşünülmüştür.
- Test ortamının gerçek dünyadaki sistemlere benzer şekilde davranarak yeterli olduğu kabul edilmiştir.

1.5. Sınırlılıklar

Bu çalışmanın bazı sınırlılıkları bulunmaktadır:

- Sadece belirli kimlik doğrulama yöntemleri analiz edilmiştir (oturum tabanlı kimlik doğrulama, JWT, OAuth 2.0, API anahtarı ve temel kimlik doğrulama).
- Çalışma yalnızca güvenlik boyutuyla ele alınmıştır ve performans veya ölçeklenebilirlik gibi faktörler dikkate alınmamıştır.
- Gerçek dünyadaki tüm sistem çeşitlilikleri ve karmaşık senaryolar çalışmaya dahil edilmemiştir.
- Kullanılan örnek uygulamalar, sadece belirli durumları yansıtmakla sınırlıdır.

1.6. Tanımlar

Bu çalışmada kullanılan temel kavramlar aşağıda tanımlanmıştır:

- **Kimlik doğrulama (Authentication):** Bir kullanıcının veya sistemin kimliğini belirlemek için yapılan süreçtir.
- **Yetkilendirme (Authorization):** Kimlik doğrulaması gerçekleşmiş bir kullanıcının hangi kaynaklara erişebileceğini belirlemektir.
- **Belirteç (Token):** Kullanıcının kimliğini doğrulamak ve erişim izinlerini tanımlamak için kullanılan dijital bilgilerdir.
- **Erişim belirteci (Access Token):** Kullanıcının kaynaklara erişim izni alması için kullanılan kısa süreli bir belirteçtir.
- **Yenileme belirteci (Refresh Token):** Erişim belirtecinin süresi dolmadan yenilenmesini sağlayan daha uzun süreli bir belirteçtir.
- **Oturum (Session):** Kullanıcının kimlik doğrulama bilgilerinin sunucuda saklandığı kimlik doğrulama yöntemidir.

BÖLÜM II

KAVRAMSAL ÇERÇEVE

2.1. Kimlik Doğrulama ve Yetkilendirme Kavramları

Kimlik doğrulama ve yetkilendirme, modern yazılım sistemlerinde güvenliğin temel yapı taşlarıdır. Kimlik doğrulama, bir kullanıcı veya sistemin kimliğini doğrulama sürecidir. Bu süreçte, kullanıcı tarafından sağlanan bilgilerin, örneğin kullanıcı adı, şifre veya belirteç gibi, doğruluğu kontrol edilir. Bu kontrolün başarılı olması, kullanıcının kimliğinin doğrulandığını gösterir; ancak bunun sisteme erişim izni verildiği anlamına gelmez. Yetkilendirme ise, kimlik doğrulaması tamamlandıktan sonra kullanıcının hangi kaynaklara erişebileceğini veya hangi işlemleri gerçekleştirebileceğini belirleme sürecidir. Bu süreçte kullanıcıya ait rolü, izinleri ve erişim hakları net bir şekilde tanımlanır. Kimlik doğrulama ve yetkilendirme kavramları bağımsız olarak da çalışabilirler, ancak sistemin güvenliğini sağlamak konusunda birlikte çalışırlar. Bu iki mekanizmanın yanlış şekilde yapılandırılması, yetkisiz erişim ve veri güvenliği ihlallerine neden olabilir. (Jones et al., 2015)

2.2. Kimlik Doğrulama Yöntemleri

Web sistemlerinde kullanıcı kimliklerinin doğrulanması, farklı tekniklerle sağlanmaktadır. Bu teknikler, sistemin yapısı, kullanıcının cihaz türü ve güvenlik ihtiyaçlarına göre değişiklik gösterebilir.

2.2.1. Oturum Tabanlı Kimlik Doğrulama (Session-Based Authentication)

Oturum tabanlı kimlik doğrulamada, kullanıcı sisteme giriş yaptıktan sonra sunucu tarafından bir oturum oluşturulur. Bu oturumla ilgili bilgiler, genellikle istemci tarafında bulunan bir çerezde tutulan oturum tanımlayıcısı sayesinde her istekte sunucuya gönderilir. Sunucu, bu bilgilerle kullanıcıyı tanır ve onu yetkilendirir. Bu yaklaşımın önemli bir avantajı, hassas bilgilerin sunucu üzerinde saklanması sayesinde daha fazla güvenlik sağlamasıdır. Ancak, oturum bilgilerinin sunucuda tutulması, özellikle büyük sistemlerde ölçeklenebilirlik açısından bazı sorunlara neden olabilir. (Hardt, 2012)

2.2.2. JSON Web Token (JWT)

JSON Web Token (JWT), istemci ve sunucu arasında güvenli veri aktarımı sağlayarak durumsuz (stateless) bir kimlik doğrulama yöntemidir. JWT'nin yapısı başlık (header), içerik (payload) ve imza (signature) olmak üzere üç bölümden oluşur. Kullanıcı sistemde kimlik doğrulaması yapıldıktan sonra bir token oluşturulur ve bu token istemci cihazında saklanır. Her istekte sunucuya gönderilerek kullanıcı kimlik bilgileri tekrar kontrol edilir. JWT'nin en önemli avantajlarından biri, sunucu üzerinde oturum bilgisi tutulmadan kimlik doğrulama işleminin yapılabilmesidir. Bu özellik, özellikle mikro servis tabanlı sistemlerde büyük ölçeklerde çalışan uygulamalarda daha yüksek esneklik ve performans sağlar. Ancak belirtecin istemci tarafında saklanması, bu token'ın yanlış kullanıma veya hırsızlığa uğraması gibi potansiyel güvenlik risklerine neden olabilir. (Jones et al., 2015; Shingala, 2019)

2.2.3. OAuth 2.0

OAuth 2.0, kullanıcıların kendi kimlik bilgilerini paylaşmadan üçüncü taraf uygulamalara erişim izni vermek için kullanılan bir yetkilendirme protokolüdür. Bu sistem, özellikle sosyal medya platformları ve dış hizmetlerle entegrasyon alanlarında yaygın şekilde uygulanmaktadır. OAuth 2.0, erişim belirteçleri aracılığıyla çalışır ve bu sayede kullanıcı verilerine güvenli bir şekilde erişim sağlanır. Ancak, bu protokolün karmaşık yapısı nedeniyle yanlış kullanıldığında sistemde güvenlik açıklarına sebep olabilir. (Hardt, 2012; Fotiou et al., 2021)

2.2.4. API Anahtarı (API Key)

API anahtar yöntemi, belirli bir API'ye istemci erişimini kontrol etmek amacıyla kullanılan basit bir kimlik doğrulama mekanizmasıdır. Bu yöntemde, her bir istemciye tekil bir anahtar verilir ve bu anahtar, her istekte sunucuya iletilir. Uygulama süreci kolay olmasına rağmen, bu yöntemde en büyük eksiklik, API anahtarlarının kötü niyetli kişilerce ele geçirilmesi halinde yetkisiz erişim riskidir. Bu yüzden bu yöntem genellikle güvenlik düzeyi düşük olan sistemlerde tercih edilir. (Gopal, 2025)

2.2.5. Basic Authentication

Temel Kimlik Doğrulama (Basic Authentication), kullanıcı adı ve şifre bilgilerinin Base64 formatında kodlanarak sunucuya iletilmesi işlevini yerine getiren basit bir kimlik doğrulama yöntemidir. Bu yöntem, HTTPS kullanılmadığında bilgilerin güvenli olmamasına neden olabilir ve bu nedenle ciddi güvenlik risklerine yol açabilir. Günümüzde genellikle test ortamlarında veya güvenlik gereksinimleri düşük olan sistemlerde tercih edilirken, modern uygulamalar için en uygun çözüm olarak kabul edilmez. (Ahmed & Mahmood, 2019)

2.3. Token Yapıları

Token temelli kimlik doğrulama yöntemlerinde, kullanıcıların kimliklerinin ve yetki düzeylerinin token'lar ile doğrulanması sağlanmaktadır. (De Cock et al., 2005)

2.3.1. Access Token

Erişim belirteci, bir kullanıcıya belirli bir süre boyunca sistem kaynaklarına erişim izni veren kısa ömürlü bir belirteç türüdür. Bu belirteç, kullanıcının izinlerini doğrulamak amacıyla her istekte sunucuya gönderilir. Kısa ömürlü olması, sistem güvenliğine katkı sağlar; ancak belirtecin süresi dolduğunda kullanıcıya yeniden doğrulama istemi gönderilmesi, kullanıcı deneyimini olumsuz etkileyebilir.

2.3.2. Refresh Token

Yenileme belirteci, geçersiz hale gelen erişim belirtecini yenilemek amacıyla kullanılan uzun ömürlü bir belirteç türüdür. Bu belirteç, güvenli bir şekilde saklanmalı ve doğrudan API'ye erişim sağlayacak şekilde kullanılmamalıdır. Yenileme belirtecinin ele geçirilmesi, saldırganın uzun süreli bir şekilde erişim sağlayabilmesine yol açabilecek önemli bir güvenlik tehdididir.

2.4. İstemci Türleri

Farklı istemci türlerinin, kimlik doğrulama yöntemlerinin uygulanmasıyla ilgili farklı ihtiyaçları vardır.

2.4.1. Web Uygulamaları

Web uygulamaları genellikle çerezlerle veya JWT'lerle sağlanan oturum yönetimi yöntemlerini kullanır. Tarayıcı güvenlik mekanizmaları, örneğin SameSite çerezleri ve CORS gibi özellikler, bu tür uygulamaların güvenliğini doğrudan etkiler.

2.4.2. Mobil Uygulamalar

Mobil uygulamalarda belirteçler genellikle cihazın güvenli depolama alanlarında tutulur. Ancak cihaz güvenliği ve tersine mühendislik gibi riskler, mobil uygulamalarda ek güvenlik adımlarının alınmasına neden olmaktadır.

2.4.3. Masaüstü Uygulamaları

Masaüstü uygulamalarında kimlik doğrulama işlemleri genellikle yerel olarak depolanan veriler kullanılarak yapılır. Bu yaklaşım, işletim sisteminin güvenlik seviyesine göre farklı türde riskler doğurabilir.

2.4.4. Komut Satırı Uygulamaları (CLI)

Komut satırı uygulamalarında kimlik doğrulama, genellikle API anahtarları ya da belirteçler aracılığıyla yapılır. Bu tür uygulamalarda belirteçlerin dosya sisteminde depolanması, güvenlik açısından bazı riskler doğurabilir.

2.5. Güvenlik Tehditleri

Kimlik doğrulama ve yetkilendirme sistemleri, farklı türdeki güvenlik tehditlerine karşı savunmasızlık gösterme riski taşıyabilir.

2.5.1. Cross-Site Scripting (XSS)

Siteler arası komut dosyası çalıştırma (XSS) saldırıları, kötü amaçlı kodların istemci tarafında çalıştırılması yoluyla kullanıcı verilerinin ele geçirilmesini sağlayabilir. Bu tür saldırılar, özellikle belirteçler tarayıcıda saklandığında ciddi güvenlik tehditlerine neden olabilir. (Gupta & Gupta, 2017)

2.5.2. Cross-Site Request Forgery (CSRF)

CSRF saldırıları, kullanıcıların bilgisi dışında yetkisiz işlemlerin yapılmasına neden olur. Bu tür saldırılar, özellikle oturum tabanlı kimlik doğrulama sistemlerinde sıkça görülen bir tehdit olabilir. (Li et al., 2018)

2.5.3. Token Hırsızlığı (Token Theft)

Belirteçlerin (token'ların) güvenliği zayıflarsa, bir saldırgan sistemin kaynaklarına yetkili bir kullanıcı gibi erişebilir. Bu nedenle, belirteçlerin saklanma ve iletim yöntemlerinin güvenli olmasının büyük önemi vardır.

2.5.4. Replay Attack

Tekrar oynatma saldırısı, mevcut bir isteğin tekrar gönderilmeye çalışmasıyla karakterize edilen bir saldırı türüdür. (De Cock et al., 2005)

BÖLÜM III

YÖNTEM

3.1. Araştırmanın Modeli

Bu çalışma, farklı istemci türleri tarafından kullanılan web API'lerindeki kimlik doğrulama ve yetkilendirme modellerinin güvenlik yönlerini değerlendirmek amacıyla tasarlanmış bir karşılaştırmalı ve uygulamalı araştırmadır. Çalışmada, oturum tabanlı kimlik doğrulama, JSON Web Token (JWT), OAuth 2.0 ve API anahtarları gibi çeşitli kimlik doğrulama yöntemlerinin farklı türde istemciler üzerindeki etkileri incelenmiştir.

Bu çalışma kapsamında, gerçek dünya kullanım senaryolarını temsil edecek şekilde bir web API'si geliştirilmiştir ve bu API'ye erişim sağlamak amacıyla farklı istemci uygulamaları oluşturulmuştur.

Bu yaklaşım sayesinde, teorik olarak analiz edilen kimlik doğrulama yöntemlerinin uygulamalı ortamda test edilmesi ve karşılaştırmalı güvenlik değerlendirmesi yapılmıştır.

Araştırma, nitel bir yöntemoloji ile yürütülmüş ve elde edilen bulgular, güvenlik riskleri, uygulanabilirlik ve istemci uyumluluğu açısından ayrıntılı bir şekilde analiz edilmiştir.

3.2. Evren ve Örneklem

Bu araştırmanın kapsamı, web API'leri kullanarak hizmet sunan ve farklı türde istemcilerin kullanımına açık olan modern yazılım sistemlerini içermektedir. Bu sistemler, genellikle mikro hizmet mimarisi ve birden fazla istemci yapısı temel alınarak değerlendirilmektedir.

Araştırmanın örnekleme, bu çalışmada geliştirilen bir örnek web API'si ile bu API'ye erişen çeşitli istemci uygulamalarını içerir.

Geliştirilen istemci uygulamaları, bir web uygulaması (React), bir mobil uygulama (React Native), bir masaüstü uygulaması ve bir komut satırı uygulaması (CLI) şeklinde tasarlanmıştır. Bu örneklem, farklı istemci türlerinin kimlik doğrulama süreçlerinde nasıl davranabileceğini karşılaştırmalı bir şekilde incelemek amacıyla seçilmiştir.

3.3. Veri Toplama Araçları

Araştırmada, veri toplama amacıyla geliştirilen yazılım sistemleri kullanılmıştır. Bu bağlamda, kimlik doğrulama ve yetkilendirme süreçlerini destekleyen temel bir bileşen olarak .NET platformunda bir web API'si tasarlanmıştır. API'ye erişim sağlayan istemci uygulamaları farklı teknolojilerle geliştirilmiştir. Web istemcisi React, mobil istemci React Native ve komut satırı istemcisi ise Rust programlama diliyle oluşturulmuştur. Bu istemciler üzerinden farklı kimlik doğrulama yöntemleri test edilmiştir. Veri toplama süreci sırasında, kimlik doğrulama işlemlerinde belirteç oluşturma, saklama ve aktarım süreçleri analiz edilmiştir. Ayrıca sistemin, çeşitli güvenlik tehditlerine karşı gösterdiği davranış da incelemeye tabi tutulmuştur.

3.4. Verilerin Toplanması

Test senaryoları, veriler, API'ler ve istemci uygulamaları üzerinden toplanmıştır. Bu kapsamda, kullanıcı girişi, belirteç oluşturma ve yenileme işlemlerinin yanı sıra API'ye erişim süreçleri, farklı istemci türleri için ayrı ayrı uygulanmıştır. Güvenlik açısından saldırı senaryoları da oluşturulmuş ve bu senaryolarda belirteç hırsızlığı, XSS saldırıları, CSRF saldırıları ve tekrar oynatma saldırıları yer almıştır. Her bir senaryoda, ilgili kimlik doğrulama yönteminin nasıl davrandığı gözlemlenmiş ve elde edilen veriler kaydedilmiştir. Daha sonra, toplanan veriler sistem davranışları ve güvenlik açıklarının tespiti açısından değerlendirilmiştir.

AHMET YESEVİ
ÜNİVERSİTESİ

3.5. Verilerin Analizi

Elde edilen bilgiler, karşılaştırmalı analiz yöntemleriyle değerlendirilmiştir. Bu değerlendirme sürecinde, her bir kimlik doğrulama yöntemi; güvenlik, istemci uyumluluğu ve uygulanabilirlik kriterlerine göre incelemeye tabi tutulmuştur. Güvenlik analizi kapsamında, her yöntemin XSS, CSRF, belirteç hırsızlığı ve tekrar oynatma saldırıları gibi tehditlere karşı direnç düzeyi incelenmiştir. İstemci uyumluluğu analizi ise, ilgili yöntemin web, mobil, masaüstü ve komut satırı arayüzü (CLI) uygulamalarında kullanımına ilişkin kolaylık ve uygunlukları değerlendirmeye tabi tutulmuştur. Elde edilen analiz sonuçları, farklı kimlik doğrulama yöntemlerinin avantaj ve dezavantajlarını ortaya koyarak karşılaştırmalı bir şekilde sunulmuştur. Bu bulgulara göre öneriler geliştirilmiştir.



BÖLÜM IV

BULGULAR VE YORUM

4.1. Kimlik doğrulama yöntemlerine ilişkin bulgular

Araştırmanın ilk alt probleminin kapsamında, farklı kimlik doğrulama yöntemlerinin web API güvenliği açısından sağladığı avantajlar ve sınırlamalar incelenmiştir. Bu çalışma kapsamında geliştirilen demo sistemde Session-Based Authentication, JWT, API Key, Basic Authentication ve OAuth benzeri bir yapı uygulanmış, istemci davranışları gözlemlenmiştir. Elde edilen bulgulara göre, oturum tabanlı kimlik doğrulama yöntemi web istemcilerinde doğal tarayıcı davranışlarıyla uyumlu çalışmaktadır.

Session bilgisi HttpOnly cookie içinde saklandığı için JavaScript üzerinden erişim engellenmiş olup bu da XSS saldırılarıyla ilgili token okuma riskini azaltmaktadır. Ancak bu bilgilerin tarayıcı tarafından otomatik olarak gönderilmesi, CSRF saldırıları açısından ciddi bir tehdit oluşturmaktadır. Çalışmada geliştirilen CSRF simülasyonu sonucunda, ek koruma önlemi alınmaması halinde session tabanlı kimlik doğrulama sistemlerinde yetkisiz işlemlerin gerçekleştirilebileceği gözlemlenmiştir.

JWT tabanlı kimlik doğrulama yöntemi, web, mobil, masaüstü ve CLI istemcileri arasında yaygın şekilde kullanılabilir. Bu da JWT yaklaşımının istemci bağımsız ve taşınabilir bir yapı sunduğunu göstermektedir. Ancak JWT, bearer token mantığıyla çalıştığı için token ele geçirildiğinde sistem tarafından geçerli kimlik bilgisi olarak kabul edilmektedir. Özellikle web istemcilerinde JWT'ın localStorage içinde saklanması, XSS ve token sızıntısı risklerini artırmaktadır.

API Key yöntemi, CLI ve servisler arası iletişim senaryolarında kullanımı kolay ve hızlı olmaktadır. Ancak API key bilgisi uzun süreli kullanıldığında ve istemci tarafında saklandığında sistemde önemli güvenlik riskleri doğmaktadır. Çalışmada kullanılan hardcoded API key yaklaşımı, gerçek sistemlerde oluşabilecek secret sızıntısı risklerinin nasıl oluşabileceğini göstermek amacıyla tercih edilmiştir.

Basic Authentication yöntemi uygulamaları en basit yöntemlerden biri olmakla birlikte, güvenlik açısından en zayıf yapılar arasında yer almaktadır. Kullanıcı adı ve parola her istekte tekrar gönderilmesi, özellikle HTTPS kullanılmayan sistemlerde kullanıcı bilgilerinin sızmasına neden olmaktadır.

OAuth 2.0 açısından değerlendirildiğinde, çalışmada gerçek OAuth altyapısı uygulanmak yerine kavramsal bir demo akışı tercih edilmiştir. Bu nedenle PKCE, scope, refresh token ve authorization server gibi gelişmiş güvenlik mekanizmaları sistemde bulunmamaktadır. Ancak oluşturulan yapı, authorization code mantığının temel çalışma prensibini göstermek açısından yeterli düzeyde değerlendirilmiştir.

4.2. İstemci türlerine ilişkin bulgular

Araştırmanın ikinci alt problemi kapsamında, farklı istemci türlerinde token saklama ve iletim yöntemlerinin güvenlik üzerindeki etkileri incelenmiştir. Çalışma kapsamında web, mobil, masaüstü ve CLI istemcileri geliştirilmiş ve aynı API üzerinde test edilmiştir.

Web istemcisinde, session tabanlı yapı için cookie tabanlı saklama yöntemi kullanılmıştır. JWT tabanlı yapıda ise token bilgisi localStorage içerisinde tutulmuştur. Yapılan incelemeler sonucunda, localStorage kullanımının XSS saldırıları açısından önemli bir risk oluşturduğu tespit edilmiştir. Özellikle istemci tarafında çalışan kötü niyetli JavaScript kodlarının, localStorage içindeki token bilgilerine erişebildiği görülmüştür. Buna karşılık HttpOnly cookie yaklaşımının, JavaScript erişimini sınırladığı tespit edilmiştir. Mobil istemcide token bilgisi React state üzerinde tutulmuştur.

Bu yaklaşım, web istemcilerindeki localStorage kullanımına göre daha düşük güvenlik riski oluşturmaktadır. Ancak güvenli mobil saklama alanları kullanılmadığı için, token bilgilerinin uygulama belleği üzerinden ele geçirilme ihtimali tamamen ortadan kalkmamıştır.

Masaüstü istemcisinde token bilgisi Electron renderer sürecinde JavaScript değişkeni olarak tutulmuştur. Token bilgisinin DOM üzerinde görüntülenmesi, istemci tarafındaki render sürecinde betik çalıştırma zafiyetlerinde token theft riskini artırır. Electron mimarisinde güvenli preload API, context isolation ve CSP gibi mekanizmalar

kullanılmaması durumunda, masaüstü istemcilerin web uygulamalarına benzer riskler taşıdığı görülmüştür.

CLI istemcisi, tarayıcı tabanlı XSS ve CSRF saldırılarından daha az etkilenmektedir. Ancak token ve API key bilgilerinin environment değişkenleri veya komut satırı argümanları üzerinden taşınması, terminal geçmişi ve process listesi kaynaklı sızıntı riskleri oluşturur. Ayrıca istemcinin sadece HTTP desteği sunması nedeniyle, ağ üzerinden dinleme saldırılarına karşı savunmasız olduğu belirlenmiştir.

Elde edilen bulgular, güvenlik risklerinin yalnızca kullanılan kimlik doğrulama yöntemine değil, token'ın hangi ortamda saklandığına ve nasıl iletildiğine de doğrudan bağlı olduğunu göstermektedir.

4.3. Güvenlik tehditlerine ilişkin bulgular

Araştırmanın üçüncü alt problemi kapsamında, JWT, Session-Based Authentication, API Key ve Basic Authentication gibi yöntemlerin XSS, CSRF, token theft ve replay attack saldırılarına karşı nasıl davrandığı incelenmiştir. XSS saldırıları açısından en yüksek riskin web istemcisinde yer alan localStorage'a kaydedilen JWT yapısında ortaya çıktığı görülmüştür. Çalışma kapsamında geliştirilen örnek senaryolarda, istemci tarafında çalışan JavaScript kodlarının token bilgilerine ulaşabildiği gözlemlenmiştir. Session tabanlı yapılar için ise HttpOnly cookie kullanımı nedeniyle doğrudan cookie okuma işlemi engellenmiştir.

CSRF saldırıları açısından en riskli yöntemin Session-Based Authentication olduğu belirlenmiştir. Tarayıcının otomatik olarak cookie bilgilerini göndermesi nedeniyle ek güvenlik mekanizmaları kullanılmadığında, kullanıcı adına yetkisiz işlemler gerçekleştirilebildiği tespit edilmiştir. SameSite=Lax kullanımı belli bir koruma sağlasa da, bu tek başına yeterli olmadığı değerlendirilmiştir. Token theft açısından JWT ve API Key yapılarının benzer riskleri taşıdığı gözlemlenmiştir. Ele geçirilen bearer token veya API key bilgilerinin sistem tarafından geçerli kimlik bilgisi olarak kabul edildiği belirlenmiştir. Özellikle uzun süreli token kullanımının saldırı etkisini artırdığı görülmüştür.

Replay attack senaryolarında aynı token veya request identifier bilgilerinin tekrar kullanılması mümkün olduğu tespit edilmiştir. Sistem tekrar eden istekleri belirleyebilmektedir ancak doğrudan engellememektedir. Bu durum, nonce, timestamp ve request signing gibi ek güvenlik mekanizmalarının kullanımını zorunlu kılmaktadır.

Basic Authentication yöntemi, credential güvenliği açısından en riskli yöntemlerden biri olarak değerlendirilmiştir. Kullanıcı adı ve parola bilgilerinin her istekte yeniden gönderilmesi, güvenli bir iletişim kanalı kullanılmadığında ciddi güvenlik açıklarına yol açmaktadır.

Araştırma sonuçları genel olarak değerlendirildiğinde, modern çoklu istemci mimarilerinde sadece bir kimlik doğrulama yönteminin tüm senaryolara karşı yeterli olmadığı, istemci türüne ve tehdit modeline göre hibrit bir güvenlik yaklaşımının tercih edilmesi gerektiği sonucuna varılmıştır.



BÖLÜM V

SONUÇ, TARTIŞMA VE ÖNERİLER

5.1. Sonuç

Bu çalışmanın amacı, farklı türde istemcilerin kullanmakta olduğu web API'lerde kimlik doğrulama ve yetkilendirme modellerini güvenlik açısından karşılaştırmalı olarak incelemektir. Çalışma kapsamında geliştirilen web, mobil, masaüstü ve komut satırı (CLI) istemcileri üzerinden, session tabanlı kimlik doğrulama, JWT, API Key, Basic Authentication ve OAuth benzeri kimlik doğrulama akışları test edilmiştir.

Elde edilen sonuçlara göre session tabanlı kimlik doğrulama, web istemcileri için doğal ve yönetilebilir bir yapı sunmaktadır. HttpOnly cookie kullanımı, XSS saldırısına karşı koruma sağlayabilmekle birlikte, CSRF saldırılarına karşı ek güvenlik önlemlerinin gerekliliği gözlemlenmiştir.

JWT yaklaşımı, istemci bağımsız ve taşınabilir bir yapı sunmaktadır.

Mobil, masaüstü ve CLI istemcilerinde ortak şekilde kullanılabilmesi önemli bir avantaj oluşturur. Ancak token bilgilerinin istemci tarafında güvenli olmayan ortamlarda saklanması durumunda token theft ve replay attack risklerinin arttığı tespit edilmiştir.

API Key yaklaşımı, servisler arası iletişime ve CLI uygulamalarında pratik kullanım sunmaktadır. Ancak hardcoded veya uzun ömürlü key kullanımı ciddi güvenlik zafiyetlerine neden olmaktadır.

Basic Authentication yöntemi, modern sistemler açısından sınırlı bir güvenlik seviyesi sunmaktadır. Özellikle her istekte kullanıcı adı ve şifre taşınması, güvenli bir iletişim kanalı kullanılmadığında, kimlik bilgileri sızıntısı riskini önemli ölçüde artırır.

Sonuç olarak, modern çoklu istemci mimarilerinde istemci türüne uygun güvenlik stratejisinin seçilmesinin kritik bir öneme sahip olduğu belirlenmiştir.

Tek bir kimlik doğrulama yaklaşımının tüm istemci türleri için yeterli güvenlik sağlayamayacağı tespit edilmiştir.

5.2. Tartışma

Araştırma bulguları, literatürde yer alan modern web güvenliği yaklaşımlarıyla oldukça uyumludur. JWT tabanlı yapıların stateless mimari avantajı sağlamasına rağmen, bearer token mekanizması nedeniyle token ele geçirilmesi durumunda büyük güvenlik risklerine yol açtığı ortaya konmuştur. Özellikle web istemcilerinde localStorage kullanımının XSS saldırı riskini artırdığı bu bulgu, literatürdeki modern web güvenliği yaklaşımlarını desteklemektedir.

Session tabanlı yapıların, web istemcilerinde daha güvenli cookie politikalarıyla birlikte kullanılmasının mümkün olduğu belirlenmiştir. Bununla birlikte, CSRF saldırılarının hâlâ önemli bir tehdit oluşturmaya devam ettiği saptanmıştır. Çalışma kapsamında kullanılan SameSite=Lax yaklaşımının tek başına yeterli koruma düzeyi sunmadığı düşünülmüştür.

Mobil ve masaüstü istemcilerde token saklama yöntemlerinin güvenlik açısından kritik bir önem taşıdığı görülmüştür. Özellikle güvenli saklama alanları kullanılmayan istemcilerde token theft riskinin arttığı belirlenmiştir. Bu durum, istemci tarafı güvenliğinin API güvenliğinin ayrılmaz bir parçası olduğunu göstermektedir.

CLI istemcilerinin tarayıcı tabanlı saldırılara göre daha dirençli olduğu görülmüş olmasına rağmen, environment değişkenleri, terminal geçmişi ve ağ iletişimi nedeniyle riskler taşıdığı tespit edilmiştir. Bu bulgu, istemci türüne bakılmaksızın secret yönetiminin çok büyük önem taşıdığını ortaya koymaktadır.

Araştırma kapsamında elde edilen bulgular, proje amaçları doğrultusunda tutarlı olarak değerlendirilmiştir. Çalışma, farklı istemci türlerinde kullanılan kimlik doğrulama yöntemlerinin güvenlik davranışlarını pratiğe uygulayarak göstermiş ve farklı tehdit senaryolarının istemci bazında nasıl değiştiğini açık şekilde ortaya koymuştur.

5.3. Öneriler

Araştırma sonuçlarına göre geliştiricilerin üretim ortamlarında doğrudan kodlarda (hardcoded) gizli bilgiler, kullanıcı verileri ve API anahtarları kullanmaktan kaçınmaları önerilmektedir. Kimlik doğrulama süreçlerinde standart güvenlik kütüphaneleri ve merkezi ortak ortam yapıları kullanılması, güvenlik yönetimini daha kolay hale getirecektir.

Web istemcilerinde JWT kullanımı sırasında bilgilerin localStorage yerine daha güvenli bir şekilde saklanması önerilmektedir. Session tabanlı sistemlerde ise HttpOnly, Secure ve uygun SameSite ayarları ile birlikte CSRF token mekanizmasının kullanılması zorunludur.

Mobil istemcilerde token bilgilerinin platforma özgü güvenli saklama alanlarında tutulması önerilmektedir. Aynı zamanda bu bilgilerin kullanıcı arayüzünde veya uygulama loglarında görünmemesine dikkat edilmelidir.

Masaüstü istemcilerinde Electron render sürecinde token bilgilerinin DOM üzerinde tutulmaması tercih edilmelidir. Context isolation, CSP ve güvenli preload API mekanizmalarının kullanılması güvenlik açısından önem taşımaktadır.

CLI istemcilerinde token ve API anahtar bilgilerinin komut satırı argümanları yerine güvenli secret yönetim mekanizmalarıyla saklanması önerilmektedir. Ayrıca istemci ve API arasında iletişim sırasında HTTPS kullanımının zorunlu hale getirilmesi gerekmektedir.

Gelecekteki çalışmalarda gerçek OAuth 2.0 Authorization Code + PKCE akışı uygulanmalı, refresh token rotation mekanizmaları eklenmeli, rol tabanlı yetkilendirme yapıları geliştirilmeli ve otomatik saldırı testleri yapılmalıdır.

KAYNAKÇA

Ahmed, S., & Mahmood, Q. (2019, November). An authentication based scheme for applications using JSON web token. In 2019 22nd international multitopic conference (INMIC) (pp. 1-6). IEEE.

De Cock, D., Wouters, K., Schellekens, D., Singelee, D., & Preneel, B. (2005, January). Threat modelling for security tokens in web applications. In *Communications and Multimedia Security: 8th IFIP TC-6 TC-11 Conference on Communications and Multimedia Security*, Sept. 15–18, 2004, Windermere, The Lake District, United Kingdom (pp. 183-193). Boston, MA: Springer US.

Fotiou, N., Siris, V. A., & Polyzos, G. C. (2021, July). Capability-based access control for multi-tenant systems using OAuth 2.0 and Verifiable Credentials. In *2021 International Conference on Computer Communications and Networks (ICCCN)* (pp. 1-9). IEEE.

Gopal, S. (2025). Building a robust OAuth token based API Security: A High level Overview. *arXiv preprint arXiv:2507.16870*.

Gupta, S., & Gupta, B. B. (2017). Cross-Site Scripting (XSS) attacks and defense mechanisms: classification and state-of-the-art. *International Journal of System Assurance Engineering and Management*, 8(Suppl 1), 512-530.

Hardt, D. (2012). The OAuth 2.0 authorization framework (No. rfc6749).

Jones, M., Bradley, J., & Sakimura, N. (2015). Json web token (jwt) (No. rfc7519).

Kristol, D., & Montulli, L. (2000). HTTP state management mechanism (No. rfc2965).

Li, W., Mitchell, C. J., & Chen, T. (2018). Mitigating CSRF attacks on OAuth 2.0 and OpenID connect. *arXiv preprint arXiv:1801.07983*.

Shingala, K. (2019). Json web token (jwt) based client authentication in message queuing telemetry transport (mqtt). *arXiv preprint arXiv:1903.02895*.

EKLER

EK A: Proje Kaynak Kodu ve Uygulama Bağlantısı

Bu çalışma kapsamında geliştirilen web API (.NET), web uygulaması (React), mobil uygulama (React Native), masaüstü uygulaması ve komut satırı arayüzüne (CLI) ait kaynak kodlar aşağıdaki GitHub deposunda paylaşılmıştır:

<https://github.com/furkancemalcaliskan/TSBG-691-03>

Söz konusu depo içerisinde her istemci türüne ait uygulamalar ayrı dizinler altında (api, web, mobile, desktop, cli) organize edilmiştir. Bu yapı sayesinde farklı istemci türlerinde kullanılan kimlik doğrulama ve yetkilendirme yöntemleri karşılaştırmalı olarak incelenebilmektedir.

